Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 260 (2025) 635–644

Procedia Computer Science

www.elsevier.com/locate/procedia

Seventh International Conference on Recent Trends in Image Processing and Pattern Recognition (RTIP2R-2024)

An Intelligent Crime Surveillance Video System For Real-Time Applications

Evany Anne M[a,*], Sivakumaran N[1]

[a]Department of Computer Science and Engineering, National Institute of Technology, Tiruchirappalli 620015, Tamil Nadu, India

Abstract

The increasing frequency of violent incidents underscores the need for advanced real-time surveillance systems. This work proposes an intelligent camera system based on deep learning algorithms for crime monitoring, capable of accurately detecting violence. The system integrates YOLO for high-precision object detection, DeepSort for tracking, OpenPose for pose estimation, and LSTM networks for action classification. The goal is to create a compact and accurate device that detects hostile activities in real time, triggers an alarm, and stores the offenders' images in a database. YOLO is used to detect faces in video frames while minimizing false positives, and DeepSort tracks individuals by assigning each a unique ID, enabling continuous surveillance in crowded areas. OpenPose evaluates body positions by identifying key points and their affinities, while an LSTM network classifies actions as violent or non-violent based on posture data. When violence is detected, the system triggers an alarm and captures images, which are securely stored on a Firebase server with timestamps for easy access. This real-time, efficient, and lightweight surveillance system improves crime detection and response across various environments.

© 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Seventh International Conference on Recent Trends in Image Processing and Pattern Recognition.

Keywords: Violence Detection; Pose Estimation; Object Tracking; RNN-Based Classification

1. Introduction

Crime rates are rising globally, prompting governments to invest heavily in surveillance systems and personnel to monitor them. While these measures are important, a more efficient approach may exist. Every day, thousands of crimes occur, with hundreds happening at any given moment. To detect them, tens of thousands of CCTV cameras are deployed in public spaces, each requiring manual monitoring. Automating this process with a Human Violence Detection System can significantly reduce costs and manpower, improving both efficiency and response time.

*Corresponding author.
E-mail address: evany@nitt.edu

1877-0509 © 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Seventh International Conference on Recent Trends in Image Processing and Pattern Recognition.
10.1016/j.procs.2025.03.242

2. Literature Survey

In recent years, violent crime detection has evolved from basic optical flow methods to advanced deep learning systems[1][2]. Traditional techniques like Violence Flows (ViF) and Gaussian models of optical flow relied on statistical models and motion data to detect violent actions[3]. Modern deep learning methods have significantly improved accuracy. For instance, OpenPose now provides detailed information about human posture and motion patterns, overcoming earlier limitations that ignored temporal dynamics[4]. Deep learning models such as 3D Convolutional Neural Networks (CNNs)[11][17][19], which integrate spatial and temporal data, show substantial performance gains when hyperparameters are optimized[16][18]. Feature selection and Softmax classifiers are used to pre-process video data and classify violent actions, often leveraging pre-trained models like ResNet with single-shot detectors. Two-stream architectures further enhance action recognition by separately analyzing spatial and temporal features before merging them for improved accuracy[5][6][7]. Advanced tracking algorithms like DeepSort, combined with object detection models like YOLO, ensure efficient and precise tracking across frames[8]. OpenPose, a widely used pose estimation tool, employs a bottom-up approach that remains effective even in crowded scenes by identifying key body points[9]. Long Short-Term Memory (LSTM) networks, which can process sequential data and retain temporal context, are especially suited for classifying activities over time, making them ideal for violence detection tasks. Together, these techniques illustrate how modern machine learning algorithms are being used to detect violence in surveillance footage with greater accuracy, scalability, and reliability[15]. Methods such as streaming media analysis, benchmark testing, support vector machines, and pre-trained convolutional neural networks[12] are now commonly employed for real-time violence detection in crowded environments[10], further highlighting the efficacy of deep learning in this field.

3. Proposed Methodology

3.1. Human Detection

CNNs can only detect the presence of objects but not their locations. For object localization and tracking, models like R-CNN and YOLO are needed. R-CNN uses selective search to create 2000 regions of interest for object detection, which is time-consuming and not suited for real-time use. YOLO speeds up the process by dividing each image into an S*S grid and searching through it, making it faster than R-CNN.

3.1.1. YOLO

The pre-trained YOLO (You Only Look Once) model is used for rapid and accurate violence detection by segmenting video frames and generating bounding boxes around detected individuals. YOLO divides the image into grids (S x S), detects objects in each grid cell based on their center position, performs bounding box regression to predict object dimensions, and calculates class probabilities. The overlap between predicted and actual boxes is measured using Intersection Over Union (IoU), with a value of 1 indicating a perfect match. Bounding box regression adjusts predicted box coordinates relative to the anchor box. For each grid cell, the center coordinates (x, y), width w, and height h of the predicted bounding box are calculated as:

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log\left(\frac{w}{w_a}\right), \quad t_h = \log\left(\frac{h}{h_a}\right) \tag{1}$$

where x_a, y_a, w_a, h_a are the anchor box parameters.

The accuracy of the predicted bounding box is evaluated using IoU, which measures the overlap between the predicted and ground truth boxes:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{2}$$

IoU equals 1 when the predicted and ground truth boxes perfectly overlap.

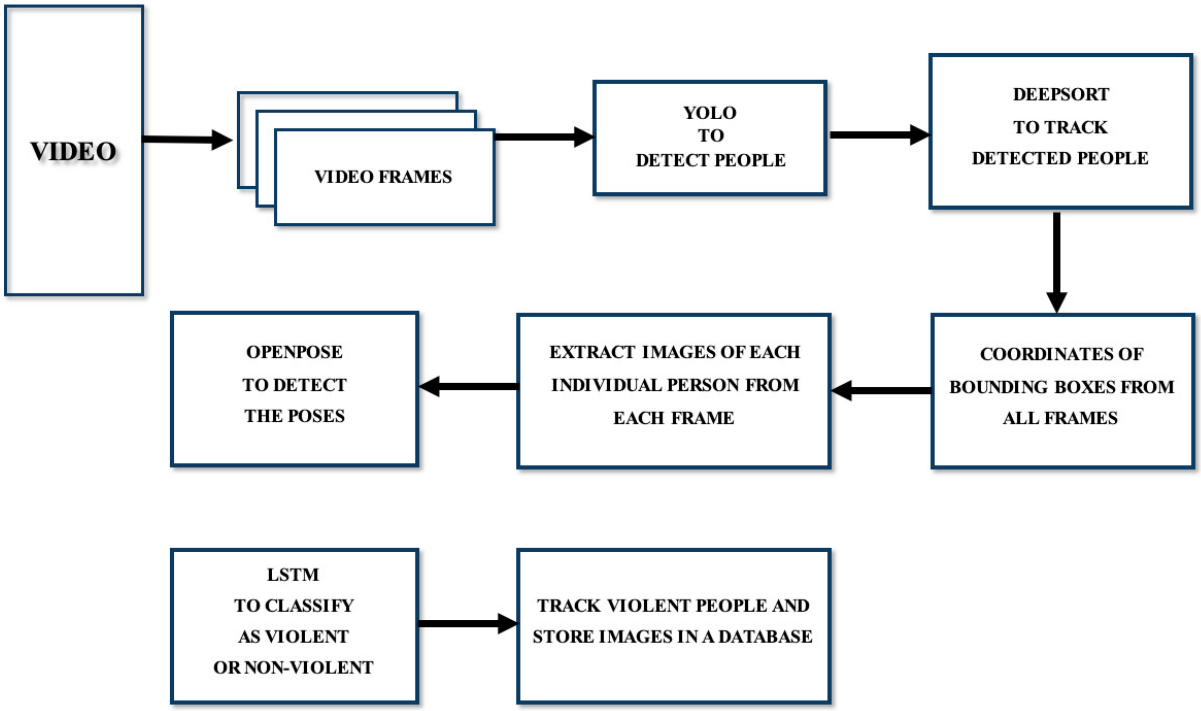

Fig. 1. Proposed Methodology

Algorithm 1 YOLO Algorithm

Input: RGB Image
Output: Image with humans detected

```
image = readImage()
N = S   ▷ We will break the Image into S coloumns and S rows and process each of the S*S different parts independently
NC = 1                                                    ▷ No. of classes
threshold = k
step = height(image)/N          ▷ step is size to take when moving across the image
pred_class_arr = new_arr(size(N,N,NCl)) ▷ stores the class for each of the S*S cells
pred_bound_box_arr = new_arr(size(NC,NC,NC,NC))
final_pred = []
for i < 0; i < NC do
    for j < 0; j < NC do
        cell = image(i:i+step,j:j+step)
        pred_class_arr[i,j] = class_pred(cell)
        pred_bound_box_arr[i,j] = bounding_box_pred(cell)
        best_b_b = [0 if pred_bound_box_arr[i,j,0,4]¿pred_bounding_box_arr[i,j,1,4] else 1]
        pred_class = ind_max_val(pred_class_arr[i,j])

        if pred_bound_box_arr[i,j,best_b_b,4]*max_value(pred_class_arr[i,j])¿k then
            preds=[pred_bound_box_arr[i,j,best_b_b,0:4], pred_class]
```

3.2. Human Tracking

DeepSORT, an enhanced tracking model, combines YOLO's coordinates with Kalman filters and metrics like Deep Appearance Descriptor and Mahalanobis Distance. It tracks human movement and reidentifies individuals after occlusions using cosine metric learning.

Algorithm 2 Tracking Algorithm

```
for everyframe do
    using the Kalman filter, make estimations for tracks.
    for every yolo detection: do
        feature=get_detection_features()
        non_Max_suppresion()
        Using anticipated Kalman sets, calculate the squared Mahalanobis.
        cosine_distance=nn_matching.NNDistMetric("cosine",max_cosine_distance,nn_budget)
    ▷ find smallest feature cosine distance for every existing tarck
        Update tracker with the use of IOUs and the Hungarian Algo

        if initiated track has been continually found for n_init frames then
            Confirm track
        elseDelete track
        end if
        if no confirmed track has been found for Max_age frames consecutively
    then
            Delete track
        end if
    end for
end for
```

Realtime Tracking in Algorithm 2 includes:

1. Detection: Identifies objects for tracking.
2. Estimation: Uses Kalman filters to estimate the target's movement.
3. Data Association: Matches detections to targets using the Hungarian algorithm and IOU distance.
4. Track Management: Creates and deletes track IDs based on IOU thresholds.

3.3. Pose Estimation

For efficient violence detection, OpenPose tracks motion over time by detecting human body key points. These pose estimates are mapped to DeepSORT IDs for continuous tracking across frames, allowing for real-time processing with reduced input data.

Algorithm 3: Pose Estimation

Input: RGB image
Extract feature maps using a two-branch multi-stage CNN.
Generate Part Confidence Maps and Part Affinity Fields from the feature maps.
Refine Confidence Maps and Affinity Fields in successive CNN stages.
Apply a greedy bipartite matching algorithm to estimate poses.
Output: Key body points for each person.

3.3.1. OpenPose

OpenPose uses Convolutional Neural Networks (CNNs) to detect human body key points and link them to tracking IDs for real-time violence detection by analyzing body angles and motion.

The OpenPose pipeline follows these steps:

1. An input RGB image is fed into a two-branch multi-stage CNN to extract feature maps.
2. The feature maps generate Part Confidence Maps, representing the likelihood of body part locations in each pixel, and Part Affinity Fields, which encode limb positions and orientations through pairwise connections between body parts.

$$S = (S_1, S_2, \ldots, S_j) \quad \text{where} \quad S_j \in \mathbb{R}^{w \times h}, \quad j \in \{1, \ldots, J\} \tag{3}$$

$$L = (L_1, L_2, \ldots, L_c) \quad \text{where} \quad L_c \in \mathbb{R}^{w \times h \times c}, \quad c \in \{1, \ldots, C\} \tag{4}$$

3. A multi-stage CNN refines the Part Affinity Fields and Confidence Maps through three steps:

   (a) Initial prediction of Part Affinity Fields.
   (b) Refinement of Confidence Maps.
   (c) Final processing using a greedy bipartite matching algorithm to obtain poses for each individual.

3.4. Matching YOLO object with pose

To match the pose estimated by OpenPose with the corresponding person, the pose coordinates are compared with YOLO's bounding box. If 70% of the pose coordinates fall within YOLO's bounding box, it is concluded that the pose and box likely belong to the same person. This matching is done using Intersection over Union (IoU), a metric that measures the overlap and area of union between the bounding boxes.

3.5. Training Neural network

To train the LSTM to detect violence, the Movie Fight Dataset and Hockey Fight Dataset were used. These datasets contain 3-5 second videos labeled as violent or non-violent. Training on these datasets enables the LSTM to classify violent actions like punching, kicking, and pouncing. Feeding pose estimates to the LSTM for testing improves performance by preventing less relevant features from influencing its decisions.

3.6. Feeding Input to the Neural Network

Using OpenPose, body part location heatmaps and part affinities are extracted from each image frame. A Gaussian filter detects heatmap peaks to identify likely positions for each body part, which are then connected to form the person's pose. These poses are linked to DeepSort detection IDs, and joint angles at the knee, shoulder, elbow, and hip are calculated for both sides. The angles are tracked over the last five frames for each individual. This sequence of angles is then fed into an LSTM model trained to detect violent behavior. For N people, the input to the LSTM is a 3D tensor of shape N58 (N: number of people, 5: past frames, 8: angles per frame).

3.7. Violence Detection

When the model detects a person as violent, the following process occurs.
1. Their bounding box changes colour from green to red so that they can be tracked easily.
2. The image of that person from each frame also goes through quality improving functions such as super-resolution and noise removal.
3. If a face is detected from any of the frames, the face and image are saved to a Firebase database.
4. An alarm is sounded to alert the authorities and law enforcement of the event.
5. The CCTV footage along with the time stamps of the violent incident is sent to the law enforcement for double checking the accuracy of the prediction.

Algorithm 4 Algorithm

```
Input: Image frames of video
Output:Video with bounding boxes for violent and non-violent people
while i < number_of_frames do
    Detections = get_yolo_detections(image_frame[i])
Tracks = DeepSort(Detections)
Human_Coords = get_human_pose_coord_using_openpose(image_frame[i])
    for track in tracks do
        for Human_Coordinate in Human_Coords do
            if overlap(track,Human_Coordinate)¿70 then
                Human[track].append(Human_Coordinate)
            end if
        end for
    end for
    Label[track] = LSTM(human[track][-5...-1])
    if label[track.ID] then
        Track.bounding_box = red
    else
        Track.bounding_box = green
    end if
end while
```

4. Results and Discussion

4.1. Datasets

The Hockey Fight dataset contains 1,000 sequences at 720×576 pixels, featuring fight and non-fight clips from National Hockey League games, with studies reporting nearly 90% accuracy in detecting fights[20]. The Movie Fight dataset [21] includes 200 fight clips from action movies, along with non-fight videos from public action recognition datasets, featuring varied scenes at different resolutions that are re-scaled to a uniform size. The RWF-2000

dataset[22], sourced from YouTube, contains 2,000 real-world surveillance videos, each cut into 5-second clips at 30 FPS and labeled as violent or non-violent, with resolutions ranging from 720P to 4K.

4.2. Results

Table 1 presents the performance comparison of our neural network across three datasets for training violence detection models. The model is evaluated using accuracy, precision, F1 score, and inference time to gauge detection speed, critical for real-time applications. Results indicate that the RWF dataset yields lower accuracy, precision, and F1 score, likely due to its realistic surveillance conditions—poor lighting, low resolution, and incomplete frames. In contrast, the hockey dataset shows better performance, likely due to favorable conditions such as consistent lighting and camera angles, along with a larger training set.

Table 1. Performance metrics for different datasets

Dataset	Avg Inference Time, s	Avg. Accuracy, %	Avg. Precision, %	Avg. F1 Score
RWF-2000	0.046	90.3	92.2	0.88
Movie Fights	0.059	94.5	95.1	0.922
Hockey Fights	0.022	96.1	97.3	0.961

4.3. Output Screenshots

4.3.1. YOLO Detection

The preliminary outcomes of person detection in video frames utilising YOLO are displayed in Figure 2. It illustrates how, whenever a human is discovered, a green bounding box is placed around the detected object of interest. A list of the coordinates for these bounding boxes is available.

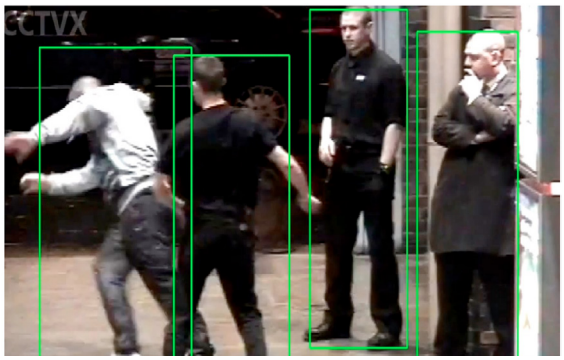

Fig. 2. Detection of humans

4.3.2. Human Tracking

Figure 3 depict the outputs of tracking individual object(human in this case) using DeepSort algorithm. The bounding boxes provided by YOLO's output are assigned with an ID number to keep track of each individual in each frame.

4.4. OpenPose Pose Estimation

The images in Figure 4 portray how our Openpose pose estimation model, identifies and represents the identified humans by their poses.

The drawn skeletons over the bodies, are accessible as a list of co-ordinates in the code. The green dots are the joints and the red lines are the relation between the joints, decided using Part affinity Fields.

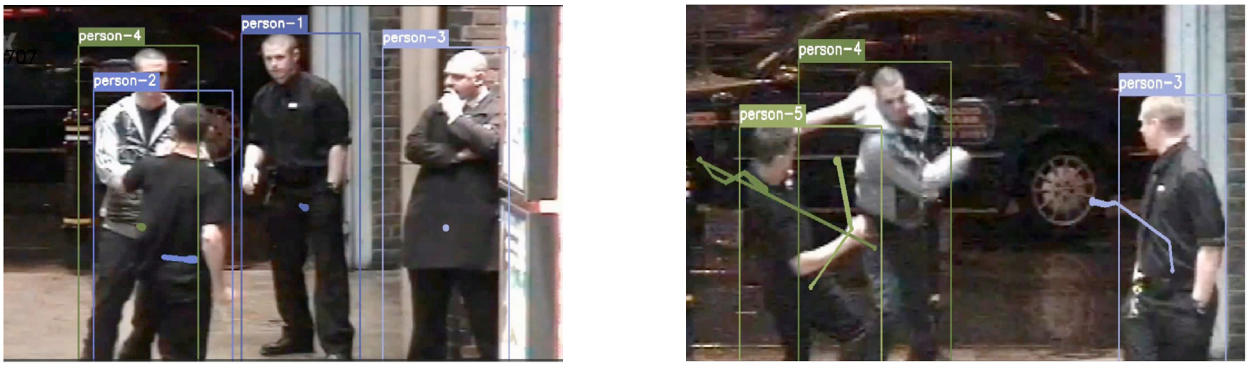

Fig. 3. Tracking of Humans

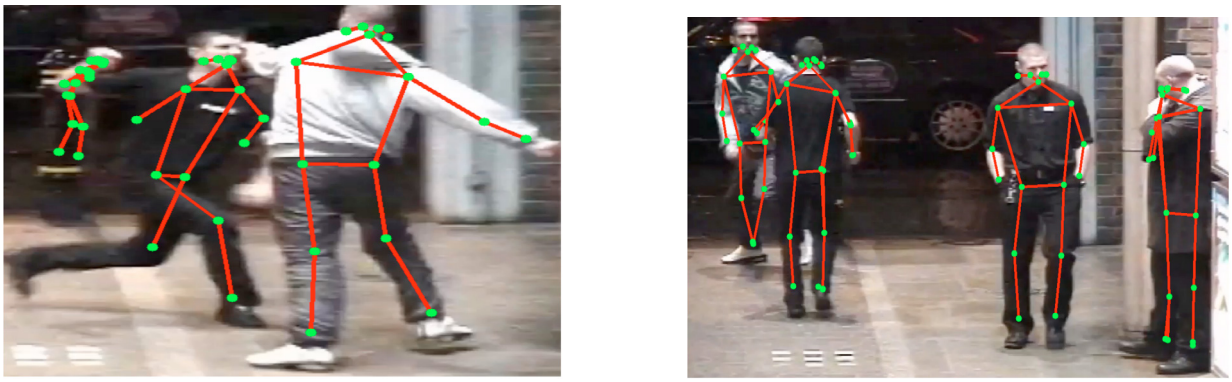

Fig. 4. Detection of Human Pose

4.5. Violence Detection

Figure 5 indicates the frames from the surveillance footage after these were passed through our final Violence Detection Model, and as can be seen it combines the results of the human tracking, pose estimation and classification by LSTM, to finally classify every identified human as violent or not. These figures illustrates that the humans bounded by a green box are non-violent and the ones bounded by red have been predicted to be violent. In the above images, the 2 people clearly engaging in a fight have both been marked red i.e. violent, and the person standing by is still green

Conclusion

This process extends the simple violence detection ability of neural networks to also locate and track violent criminals from CCTV footage. This automates the process and reduces the effort from law enforcement and security personnel who no longer need to search through long durations of footage for the violent event and track the criminals manually to save their photos. The accuracy of YOLO and DeepSort is lower than necessary and occasionally the tracker ID switches between people when they overlap and innocent people and violent people might switch markers, causing false positive and false negative detection. Therefore, right now, it is best to have human intervention and double-checking to prevent the model from accidentally accusing innocent people. But due to the modular nature of the architecture, as research happens and more modules similar to YOLO, DeepSort, and OpenPose are designed, the modules used in our paper can be switched for better ones, therefore improving the accuracy in tracking and detecting violence without needing to change the architecture too much.

Fig. 5. Detection of Violence and Tracking Violent Ones

Acknowledgement

The research work is supported by the C-DAC, Trivandrum under the project ”Next generation Emergency Response Support System”

References

[1] A. Jain, K. Gandhi, D. K. Ginoria and P. Karthikeyan, ”Human Activity Recognition with Videos Using Deep Learning,” 2021 International Conference on Forensics, Analytics, Big Data, Security (FABS), Bengaluru, India, 2021, pp. 1-5, doi: 10.1109/FABS52071.2021.9702599.

[2] K. Basu and A. M. Parimi, ”Optical Flow based method for Violence Detection,” 2023 First International Conference on Cyber Physical Systems, Power Electronics and Electric Vehicles (ICPEEV), Hyderabad, India, 2023, pp. 1-6, doi: 10.1109/ICPEEV58650.2023.10391925.

[3] P. Vashistha, C. Bhatnagar and M. A. Khan, ”An architecture to identify violence in video surveillance system using ViF and LBP,” 2018 4th International Conference on Recent Advances in Information Technology (RAIT), Dhanbad, India, 2018, pp. 1-6, doi:10.1109/RAIT.2018.8389027.

[4] B. Choi, W. An and H. Kang, ”Human Action Recognition Method using YOLO and OpenPose,” 2022 13th International Conference on Information and Communication Technology Convergence (ICTC), Jeju Island, Korea, Republic of, 2022, pp. 1786-1788, doi: 10.1109/ICTC55196.2022.9952808.

[5] C. -J. Huang, M. Gochoo and T. -H. Tan, ”Two-Stream Architecture Using RGB-based ConvNet and Pose-based LSTM for Video Action Recognition,” 2023 15th International Conference on Innovations in Information Technology (IIT), Al Ain, United Arab Emirates, 2023, pp. 127-131, doi: 10.1109/IIT59782.2023.10366415.

[6] Z. Islam, M. Rukonuzzaman, R. Ahmed, M. H. Kabir and M. Farazi, ”Efficient Two-Stream Network for Violence Detection Using Separable Convolutional LSTM,” 2021 International Joint Conference on Neural Networks (IJCNN), Shenzhen, China, 2021, pp. 1-8, doi:10.1109/IJCNN52387.2021.9534280.

[7] Y. Wan, Z. Yu, Y. Wang and X. Li, ”Action Recognition Based on Two-Stream Convolutional Networks With Long-Short-Term Spatiotemporal Features,” in IEEE Access, vol. 8, pp. 85284-85293, 2020, doi: 10.1109/ACCESS.2020.2993227.

[8] M. I. H. Azhar, F. H. K. Zaman, N. M. Tahir and H. Hashim, ”People Tracking System Using DeepSORT,” 2020 10th IEEE International Conference on Control System, Computing and Engineering (ICCSCE), Penang, Malaysia, 2020, pp. 137-141, doi: 10.1109/ICCSCE50387.2020.9204956.

[9] S. -W. Chu and C. -M. Wang, ”Combining OpenPose with BiLSTM for Violence Detection in Long-Term Care,” 2022 IEEE/ACIS 23rd International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), Taichung, Taiwan, 2022, pp. 212-215, doi:10.1109/SNPD54884.2022.10051807.

[10] T. Hassner, Y. Itcher and O. Kliper-Gross, ”Violent flows: Real-time detection of violent crowd behavior,” 2012 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops, Providence, RI, USA, 2012, pp. 1-6, doi: 10.1109/CVPRW.2012.6239348.

[11] J. Su, P. Her, E. Clemens, E. Yaz, S. Schneider and H. Medeiros, ”Violence Detection using 3D Convolutional Neural Networks,” 2022 18th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS), Madrid, Spain, 2022, pp. 1-8, doi: 10.1109/AVSS56176.2022.9959393.

[12] N. Honarjoo, A. Abdari and A. Mansouri, ”Violence detection using pre-trained models,” 2021 5th International Conference on Pattern Recognition and Image Analysis (IPRIA), Kashan, Iran, 2021, pp. 1-4, doi: 10.1109/IPRIA53572.2021.9483558.

[13] M. Perez, A. C. Kot and A. Rocha, ”Detection of Real-world Fights in Surveillance Videos,” ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Brighton, UK, 2019, pp. 2662-2666, doi: 10.1109/ICASSP.2019.8683676.

[14] M. R. Islam et al., "Crime Prediction by Detecting Violent Objects and Activity Using Pre-Trained YOLOv8n and MoViNet A0 Models," 2023 International Conference on Modeling E-Information Research, Artificial Learning and Digital Applications (ICMERALDA), Karawang, Indonesia, 2023, pp. 44-49, doi: 10.1109/ICMERALDA60125.2023.10458156.

[15] S. Chopra, L. Zhang and M. Jiang, "Human Action Recognition Using Multi-Stream Fusion and Hybrid Deep Neural Networks," 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), Honolulu, Oahu, HI, USA, 2023, pp. 4852-4858, doi: 10.1109/SMC53992.2023.10393912.

[16] F. A. Razzaq, M. A. Chaudary, S. Fareed, W. Tariq, M. Waqas and S. Javaid, "Enhancing Public Safety: Detection of Weapons and Violence in CCTV Videos with Deep Learning," 2023 25th International Multitopic Conference (INMIC), Lahore, Pakistan, 2023, pp. 1-6, doi: 10.1109/INMIC60434.2023.10465800.

[17] S. Jianjie and Z. Weijun, "Violence Detection Based on Three-Dimensional Convolutional Neural Network with Inception-ResNet," 2020 IEEE Conference on Telecommunications, Optics and Computer Science (TOCS), Shenyang, China, 2020, pp. 145-150, doi:10.1109/TOCS50858.2020.9339755.

[18] C. Vo-Le, H. S. Vo, T. D. Vu and N. H. Son, "Violence Detection using Feature Fusion of Optical Flow and 3D CNN on AICS-Violence Dataset," 2022 IEEE Ninth International Conference on Communications and Electronics (ICCE), Nha Trang, Vietnam, 2022, pp. 395-399, doi: 10.1109/ICCE55644.2022.9852065.

[19] Y. Lyu and Y. Yang, "Violence Detection Algorithm Based on Local Spatio-temporal Features and Optical Flow," 2015 International Conference on Industrial Informatics - Computing Technology, Intelligent Technology, Industrial Information Integration, Wuhan, China, 2015, pp. 307-311, doi: 10.1109/ICIICII.2015.157.

[20] Selda Ertrk. *Hockey Fight Detection Dataset*, 2020. Available at: `https://www.kaggle.com/datasets/seldaertrk/hockey-fight`. Accessed: 2024-09-29.

[21] Naveen Kumar. *Movies Fight Detection Dataset*, 2022. Available at: `https://www.kaggle.com/datasets/naveenk903/movies-fight-detection-dataset`. Accessed: 2024-09-29.

[22] Vu Lam Nguyen. *RWF-2000: Real-World Violence Detection Dataset*, 2021. Available at: `https://www.kaggle.com/datasets/vulamnguyen/rwf2000`. Accessed: 2024-09-29.